

Documentação BD e telas de Paciente

Tabela, CRUD, API, ipcHandle, ipcRenderer, service, interface de Paciente

A ordem da documentação será:

1. Estrutura da tabela e schema (src/db/**database.ts**)
2. Interface e tipos do paciente (src/types/**Paciente.ts**, src/types/**globals.d.ts**)
3. Funções de CRUD (src/db/**paciente.ts**)
4. Encapsulamento e centralização das chamadas à API exposta em **window.api.paciente** (src/renderer/services/**pacienteService.ts**)
5. Integração com o backend local via **ipcMain** (src/**index.ts**)
6. Exposição para o frontend via preload (src/**preload.ts**)

Já vou começar pela **documentação detalhada da estrutura da tabela de pacientes**, que está no arquivo **database.ts**. Em instantes te mostro!

Documentação Técnica: Tabela e Estrutura de Pacientes (**database.ts**)

 Arquivo: **src/db/database.ts**

Este arquivo é responsável por criar e gerenciar o banco de dados local **equilibra.db**, utilizando a biblioteca **better-sqlite3**. Ele define a tabela **pacientes** e garante sua existência ao inicializar o sistema.

Estrutura da Tabela **pacientes**

```
CREATE TABLE IF NOT EXISTS pacientes (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
nome TEXT NOT NULL,  
data_nascimento TEXT,  
cpf TEXT,  
telefone TEXT,  
email TEXT,  
profissao TEXT,  
estado_civil TEXT,  
criado_em TEXT DEFAULT (datetime('now','localtime')),  
atualizado_em TEXT DEFAULT (datetime('now','localtime'))  
)
```

Descrição dos Campos:

Campo	Tipo	Obrigatório	Descrição
id	INTEGER (PK)	Sim	Identificador único autoincrementável.
nome	TEXT	Sim	Nome completo do paciente.
data_nascimento	TEXT	Não	Data de nascimento, armazenada como string (idealmente em formato ISO).
cpf	TEXT	Não	CPF do paciente.
telefone	TEXT	Não	Número de telefone.
email	TEXT	Não	Endereço de e-mail.

<code>profissao</code>	TEXT	Não	Profissão atual do paciente.
<code>estado_civil</code>	TEXT	Não	Estado civil atual.
<code>criado_em</code>	TEXT	Não	Timestamp da criação, definido automaticamente.
<code>atualizado_em</code>	TEXT	Não	Timestamp da última atualização, definido automaticamente.

Lógica de Criação

O código verifica se a tabela `pacientes` já existe com:

ts

Copiar código

```
db.prepare(`
    CREATE TABLE IF NOT EXISTS pacientes (...)
`).run();
```

Esse comando garante que a tabela será criada apenas se ainda não existir, evitando sobrescrever dados.

Conexão com o Banco

A conexão é feita com:

ts

Copiar código

```
const db = new Database('equilibra.db');
```

Este banco é um arquivo `.db` local salvo no diretório raiz de execução do app.

✓ Observações Técnicas

- O uso de `datetime('now', 'localtime')` garante que os timestamps de criação e atualização estejam de acordo com o horário local do sistema.
 - Ainda não há *triggers* de atualização automática para `atualizado_em` nas edições (isso pode ser feito manualmente no CRUD).
 - O tipo `TEXT` é usado mesmo para datas, seguindo o padrão do SQLite.
-

Se estiver pronto, prossigo para a **interface e tipos de dados do paciente**, definidos em `Paciente.ts` e `globals.d.ts`. Deseja seguir?

Você disse:

Sim, prossiga, meu grande amigo.

O ChatGPT disse:

Perfeito, meu irmão. Vamos para a **documentação da interface e tipos de dados dos pacientes** conforme definidos nos arquivos `Paciente.ts` e `globals.d.ts`.

📖 Documentação Técnica: Tipos e Interfaces de Paciente

🔗 Arquivo: `src/types/Paciente.ts`

Este arquivo define a interface central que representa um paciente no sistema. Ela é usada amplamente em todo o app para garantir **tipagem estática, autocompletar e segurança de dados**.

ts

Copiar código

```
export interface Paciente {  
  id?: number;  
  nome: string;  
  data_nascimento?: string;  
  cpf?: string;  
  telefone?: string;  
  email?: string;  
  profissao?: string;  
  estado_civil?: string;  
  criado_em?: string;  
  atualizado_em?: string;  
}
```

Detalhamento dos Campos

Campo	Tipo	Obrigatório	Descrição
id	number	Não	Identificador único do paciente.
nome	string	Sim	Nome completo do paciente.
data_nascimento	string	Não	Data de nascimento no formato string (ex: ISO).
cpf	string	Não	CPF (sem validação explícita neste ponto).
telefone	string	Não	Número de telefone.

<code>email</code>	<code>string</code>	Não	Endereço de e-mail.
<code>profissao</code>	<code>string</code>	Não	Profissão atual.
<code>estado_civil</code>	<code>string</code>	Não	Estado civil (ex: solteiro, casado).
<code>criado_em</code>	<code>string</code>	Não	Timestamp de criação do registro.
<code>atualizado_em</code>	<code>string</code>	Não	Timestamp da última atualização do registro.

🧠 Observação: essa interface está perfeitamente alinhada com a estrutura da tabela no banco de dados (`database.ts`), garantindo total compatibilidade.

🌐 Arquivo: `globals.d.ts`

Aqui é feita a exposição global do tipo `Paciente` no contexto do preload (`window.api`), garantindo que o TypeScript reconheça corretamente as operações IPC entre backend (Electron) e frontend (React).

ts

Copiar código

```
export {};
```

```
declare global {  
  interface Paciente {  
    id?: number;  
    nome: string;  
    data_nascimento?: string;  
    cpf?: string;
```

```

    telefone?: string;
    email?: string;
    profissao?: string;
    estado_civil?: string;
    criado_em?: string;
    atualizado_em?: string;
  }

  interface Window {
    api: {
      paciente: {
        listar: () => Promise<Paciente[]>;
        adicionar: (paciente: Paciente) => Promise<void>;
        editar: (paciente: Paciente) => Promise<void>;
        excluir: (id: number) => Promise<void>;
        buscarPorId: (id: number) => Promise<Paciente |
undefined>;
      };
    };
  }
}

```

 **Destaques:**

- A `interface Window` expande a API do preload para incluir todas as funções de CRUD do paciente.
- Isso permite chamadas como `window.api.paciente.listar()` diretamente no React, com tipagem segura.
- O uso de `declare global` permite que o escopo global do projeto reconheça automaticamente essas interfaces, sem precisar importar manualmente em cada arquivo.

Documentação Técnica Corrigida: CRUD de Pacientes

 Caminho: `src/db/paciente.ts`

Funções CRUD Implementadas

1. Adicionar Paciente

ts

Copiar código

```
export function adicionarPaciente(paciente: Paciente): void
```

- Insere um novo paciente na tabela `pacientes`.
- Campos `criado_em` e `atualizado_em` são definidos automaticamente no SQL.

ts

Copiar código

```
INSERT INTO pacientes (  
    nome, data_nascimento, cpf, telefone, email, profissao,  
    estado_civil,  
    criado_em, atualizado_em
```



```
) VALUES (?, ?, ?, ?, ?, ?, ?, datetime('now','localtime'),  
datetime('now','localtime'))
```

2. Listar Pacientes

ts

Copiar código

```
export function listarPacientes(): Paciente[]
```

- Recupera todos os registros da tabela **pacientes**, ordenados alfabeticamente pelo **nome**.

ts

Copiar código

```
SELECT * FROM pacientes ORDER BY nome ASC
```

3. Buscar Paciente por ID

ts

Copiar código

```
export function buscarPacientePorId(id: number): Paciente |  
undefined
```

- Retorna os dados de um paciente específico.

ts

Copiar código

```
SELECT * FROM pacientes WHERE id = ?
```

4. Editar Paciente

ts

Copiar código

```
export function editarPaciente(paciente: Paciente): void
```

- Atualiza todos os campos de um paciente existente.
- Atualiza `atualizado_em` automaticamente no momento da alteração.

ts

Copiar código

```
UPDATE pacientes SET  
  
    nome = ?, data_nascimento = ?, cpf = ?, telefone = ?, email =  
    ?,  
  
    profissao = ?, estado_civil = ?, atualizado_em =  
    datetime('now', 'localtime')  
  
WHERE id = ?
```

5. Excluir Paciente

ts

Copiar código

```
export function excluirPaciente(id: number): void
```

- Exclui o registro com base no `id`.

ts

Copiar código

```
DELETE FROM pacientes WHERE id = ?
```

Integração no Projeto

- Este módulo (`src/db/paciente.ts`) atua como camada de acesso ao banco de dados local (**better-sqlite3**).
- Todas as funções são chamadas indiretamente via IPC por meio do arquivo `src/index.ts` e expostas ao frontend por `window.api` no `preload.ts`.

Documentação Técnica: Serviço de Pacientes no Renderer

 Caminho: `src/renderer/services/pacienteService.ts`

Objetivo do Arquivo

O módulo `pacienteService.ts` encapsula e **centraliza as chamadas à API exposta em `window.api.paciente`**. Essa abordagem melhora a organização e evita que chamadas brutas ao `window.api` fiquem espalhadas pelas telas.

Funções Exportadas

Cada função abaixo chama o respectivo método de `window.api.paciente`:

1. listarPacientes

ts

Copiar código

```
export const listarPacientes = () => {
```

```
return window.api.paciente.listar();  
};
```

- Retorna a lista de pacientes do banco.
 - Mapeia diretamente o método `listar()` exposto pelo preload.
-

2. adicionarPaciente

ts

Copiar código

```
export const adicionarPaciente = (paciente: Paciente) => {  
  return window.api.paciente.adicionar(paciente);  
};
```

- Envia um objeto `Paciente` para ser salvo no banco.
 - O campo `id` deve estar ausente ou indefinido.
-

3. editarPaciente

ts

Copiar código

```
export const editarPaciente = (paciente: Paciente) => {  
  return window.api.paciente.editar(paciente);  
};
```

- Atualiza os dados de um paciente com base no `id`.
-

4. `excluirPaciente`

ts

Copiar código

```
export const excluirPaciente = (id: number) => {  
  return window.api.paciente.excluir(id);  
};
```

- Remove o paciente identificado por `id`.
-

5. `buscarPacientePorId`

ts

Copiar código

```
export const buscarPacientePorId = (id: number) => {  
  return window.api.paciente.buscarPorId(id);  
};
```

- Retorna os dados completos de um único paciente.
-

Vantagens da Arquitetura

-  Isolamento das chamadas IPC do restante da aplicação React.

-  Facilidade para aplicar validações, logs ou tratamento de erros posteriormente neste nível.
-  Tipagem consistente via interface **Paciente** já declarada globalmente.

Documentação Técnica: Registro de Canais IPC – Pacientes

 Caminho: `src/index.ts`

Função Geral

O arquivo `index.ts` é o ponto de entrada da aplicação Electron no processo principal (`main process`). Além de configurar a janela principal, ele registra os **canais IPC** que respondem às requisições do frontend (renderer) feitas por `window.api`.

No caso da entidade **paciente**, este arquivo:

1. Importa as funções do módulo `src/db/paciente.ts`
 2. Cria handlers com `ipcMain.handle()` para cada operação CRUD
 3. Delega as chamadas vindas do preload
-

Trecho Relevante – Pacientes

ts

Copiar código

```
import {  
  
  listarPacientes,  
  
  adicionarPaciente,  
  
  editarPaciente,  
  
  excluirPaciente,  
  
  buscarPacientePorId,
```

```

} from './db/paciente';

ipcMain.handle('paciente:listar', () => listarPacientes());

ipcMain.handle('paciente:adicionar', (_, paciente) =>
adicionarPaciente(paciente));

ipcMain.handle('paciente:editar', (_, paciente) =>
editarPaciente(paciente));

ipcMain.handle('paciente:excluir', (_, id) =>
excluirPaciente(id));

ipcMain.handle('paciente:buscarPorId', (_, id) =>
buscarPacientePorId(id));

```

Detalhamento dos Canais

Canal IPC (<code>ipcMain.handle</code>)	Parâmetros	Função Interna	Descrição
<code>paciente:listar</code>	<code>()</code>	<code>listarPacientes()</code>	Retorna todos os pacientes.
<code>paciente:adicionar</code>	<code>(_, paciente: Paciente)</code>	<code>adicionarPaciente()</code>	Insere um novo paciente.
<code>paciente:editar</code>	<code>(_, paciente: Paciente)</code>	<code>editarPaciente()</code>	Atualiza os dados de um paciente existente.
<code>paciente:excluir</code>	<code>(_, id: number)</code>	<code>excluirPaciente()</code>	Remove o paciente com o <code>id</code> especificado.

<code>paciente:buscarP</code>	<code>(_, id:</code>	<code>buscarPacienteP</code>	Recupera os dados completos de um paciente específico.
<code>orId</code>	<code>number)</code>	<code>orId()</code>	

📌 Observação: O primeiro argumento `_` representa o `event`, que não é utilizado nesse contexto.

🧠 Conclusão Técnica

- Este arquivo é o **único ponto de entrada para todas as chamadas ao banco de dados** vindas da camada de UI.
- O uso de `ipcMain.handle` cria promessas que podem ser acessadas via `window.api` no preload, mantendo segurança e isolamento entre processos.

📖 Documentação Técnica: Exposição da API de Pacientes no `window`

📌 Caminho: `src/preload.ts`

🎯 Objetivo do Arquivo

O `preload.ts` roda em contexto isolado (context bridge) e tem como função principal:

- Expor, de forma segura e tipada, funções do `main process` (registradas via `ipcMain.handle`) ao frontend.
- Criar a API `window.api.paciente`, acessível diretamente nas páginas React.

🔒 Uso de `contextBridge` e `ipcRenderer`

ts

Copiar código

```
import { contextBridge, ipcRenderer } from 'electron';
```

🌿 Exposição da API: Pacientes

ts

Copiar código

```
contextBridge.exposeInMainWorld('api', {  
  paciente: {  
    listar: () => ipcRenderer.invoke('paciente:listar'),  
    adicionar: (paciente) =>  
ipcRenderer.invoke('paciente:adicionar', paciente),  
    editar: (paciente) => ipcRenderer.invoke('paciente:editar',  
paciente),  
    excluir: (id) => ipcRenderer.invoke('paciente:excluir', id),  
    buscarPorId: (id) =>  
ipcRenderer.invoke('paciente:buscarPorId', id),  
  },  
  // ... outras entidades como consulta, relatório etc.  
});
```

📌 Detalhamento das Funções

Método	Canal IPC Interno	Retorno Esperado
window.api.paciente		
listar()	'paciente:listar'	Promise<Pacient e[]>

<code>adicionar(paciente)</code>	<code>'paciente:adicionar'</code>	<code>Promise<void></code>
<code>editar(paciente)</code>	<code>'paciente:editar'</code>	<code>Promise<void></code>
<code>excluir(id)</code>	<code>'paciente:excluir'</code>	<code>Promise<void></code>
<code>buscarPorId(id)</code>	<code>'paciente:buscarPorId'</code>	<code>Promise<Paciente></code>

Considerações Técnicas

- ✓ Usa `contextBridge` para impedir exposição insegura da API Electron ao navegador.
 - ✓ Tipagem garantida com auxílio de `globals.d.ts`, permitindo autocomplete e segurança no TypeScript.
 - ✓ Totalmente compatível com o `pacienteService.ts` do renderer, que consome essa API.
-

✓ Fluxo Completo do CRUD de Pacientes

mermaid

Copiar código

graph TD

A[React Page] --> B[pacienteService.ts]

B --> C>window.api.paciente

C --> D[preload.ts]

D --> E[ipcRenderer.invoke()]

E --> F[index.ts com ipcMain.handle()]

F --> G[src/db/paciente.ts]

G --> H[SQLite (equilibra.db)]

Telas de interação para Pacientes (src/renderer/pages/Paciente)

Documentação Técnica: Tela Inicial de Pacientes (**index.tsx**)

 Caminho: `src/renderer/pages/Paciente/index.tsx`

Objetivo da Tela

A página `index.tsx` é a **entrada principal da seção de pacientes** no sistema Equilibra Manager. Ela apresenta um **menu visual com atalhos para funcionalidades principais**, como adicionar, listar ou filtrar pacientes.

Estrutura Visual e Funcional

Layout

- A tela usa um **plano de fundo com imagem personalizada** (`background3.png`).
- No topo, exibe o **logo de gestão de pacientes** (`gestaopacienteslogo.png`) centralizado.
- Os botões são estilizados por meio do CSS `PacienteModule.css`.

Botões principais

1. **+** Adicionar Paciente
 - Rota: `./adicionar`
 - Leva à tela de criação de um novo paciente.

2. Gerenciar pacientes ▼

- Um botão expansível que ativa um submenu.

📁 Submenu Expandido (quando `expanded === true`):

Botão	Rota	Descrição
 Listar pacientes ativos	<code>./listar?status=ativo</code>	Lista pacientes com status ativo.
 Listar pacientes pausados	<code>./listar?status=pausado</code>	Lista pacientes com status pausado.
 Listar pacientes finalizados	<code>./listar?status=finalizado</code>	Lista pacientes com status finalizado.
 Aniversariantes do mês	<code>/Pacientes/aniversariantes</code>	Lista pacientes que fazem aniversário no mês atual. (⚠️ Rota futura?)

💡 A expansão do menu é controlada por um hook `useState` com o estado booleano `expanded`.

🧠 Considerações Técnicas

- O componente usa `useNavigate()` do React Router para navegação entre páginas.
 - A estilização depende fortemente de classes definidas em `PacienteModule.css`.
 - O menu é reativo e controlado localmente, o que favorece UX simples e leve.
 - O botão de aniversariantes depende de uma rota ainda não implementada (presumivelmente `/Pacientes/aniversariantes`).
-

✓ Funções-chave

ts

Copiar código

```
const [expanded, setExpanded] = useState(false);  
const toggleExpand = () => setExpanded(!expanded);
```

Essas funções controlam o estado do menu de gerenciamento.

📖 Documentação Técnica: Tela “Adicionar Paciente”

📌 Caminho: `src/renderer/pages/Paciente/adicionar.tsx`

🎯 Objetivo da Tela

Esta página permite o cadastro de um novo paciente, preenchendo campos básicos e complementares, com uso de máscaras e validação de entrada. Ao final, os dados são enviados via `pacienteService.ts` para persistência no banco local (`better-sqlite3`).

🧩 Estrutura do Formulário

A UI é organizada dentro de um componente reutilizável chamado `FormularioAdd`, com faixa superior de navegação (`BotaoVoltar`) e plano de fundo (`background3.png`).

📋 Campos preenchíveis

Campo	Obrigatório	Tipo	Observações
Nome completo	✓	<code>TextField</code>	Livre
Data de nascimento	✓	<code>XPDateField</code>	Com máscara <code>DD/MM/AAAA</code>

Sexo	✓	Select	M, F, O
CPF	✓	XPCPFfield	Com máscara 999.999.999-99
Telefone	✓	XPPhoneField	Com máscara (99) 99999-9999
E-mail	✗	TextField	Livre
Endereço	✗	TextField	Livre
Profissão	✗	TextField	Livre
Estado civil	✗	TextField	Livre
Status	✓	Select	ativo, pausado, finalizado
Observações	✗	TextField multiline	Comentários gerais

✓ Funcionalidades principais

📦 Validação e envio

- Verifica campos obrigatórios e formato da data antes de enviar.
- Converte data de nascimento de DD/MM/AAAA para AAAA-MM-DD.

ts

Copiar código

```
const converterDataParaFormatoISO = (data: string) => {
  const [dia, mes, ano] = data.split('/');
```

```
    return `${ano}-${mes}-${dia}`;  
};
```

Objeto final enviado para salvamento

ts

Copiar código

```
const novoPaciente = {  
  nome_completo: nome,  
  telefone,  
  cpf,  
  data_nascimento: dataFormatada,  
  sexo,  
  email,  
  endereco,  
  profissao,  
  estado_civil,  
  observacoes,  
  status,  
};
```

 Envio ocorre via `await salvarPaciente(novoPaciente);`

Feedback ao usuário

- Exibe **Snackbar** com **Alert** de sucesso ou erro.

- Em caso de sucesso, redireciona para `/paciente` após 1.5s.
-

Estética e UX

- Utiliza MUI (`@mui/material`) com temas definidos em `Colors.ts`.
 - Os campos possuem espaçamento (`sx={{ mb: 2 }}`) e design adaptado para responsividade.
 - A ação de voltar está integrada ao componente fixo `BotaoVoltar`, respeitando o estado `drawerOpen`.
-

Integrações Envolvidas

- `salvarPaciente` vem de `src/renderer/services/pacienteService.ts`
- `XPDateField`, `XPCPFField`, `XPPhoneField` são componentes personalizados reutilizáveis
- O layout visual usa `FormularioCard` como estrutura de base

Documentação Técnica: Tela “Editar Paciente”

 Caminho: `src/renderer/pages/Paciente/editar.tsx`

Objetivo da Tela

Permitir a **edição dos dados de um paciente já existente** no sistema. O formulário carrega os dados com base no `id` presente na URL e permite que o usuário edite qualquer campo, salvando as alterações no banco local.

Funcionalidades de Navegação e Contexto

Função	Descrição
<code>useNavigate</code>	Usado para redirecionar para <code>/paciente/listar</code> após edição.
<code>useLocation</code> + <code>URLSearchParams</code>	Extraí o <code>id</code> do paciente a partir da URL.
<code>useOutletContext</code>	Captura o estado <code>drawerOpen</code> para exibir corretamente o botão de voltar.

Carregamento dos Dados

Busca dos dados no backend

ts

Copiar código

```
useEffect(() => {  
  const data = await buscarPacientePorId(id);  
  setPaciente(data);  
}, [id]);
```

- Chama `buscarPacientePorId` do `pacienteService.ts`.
 - O paciente carregado é armazenado em `paciente`, tipo `Paciente`.
-

Edição do Formulário

A tela mostra o formulário completo com os campos editáveis. O estado é atualizado dinamicamente com `handleChange`.

Campo	Tipo	Observações
Nome completo	TextField	<code>paciente.nome_completo</code>
CPF	XPCPFfield	Campo mascarado
Telefone	XPPhoneField	Campo mascarado
Email	TextField	Opcional
Endereço	TextField	Opcional
Profissão	TextField	Opcional
Estado civil	TextField	Opcional
Observações	TextField	Multiline
Status	Select	Opções: Ativo, Pausado, Finalizado

Salvar Alterações

Atualização no backend

ts

Copiar código

```
await atualizarPaciente(paciente);
```

- A função `handleAtualizar` envia os dados editados usando `pacienteService.ts`.

Feedback ao Usuário

Dois **Snackbars** são utilizados:

-  Sucesso → “Paciente atualizado com sucesso!”
-  Erro → “Erro ao atualizar paciente”

Em caso de sucesso, o usuário é redirecionado após 1.5s.

Considerações Técnicas

- O componente depende da interface **Paciente** tipada com campos opcionais.
- O botão “Voltar” é gerenciado por **BotaoVoltar**, e o layout segue o padrão com **FormularioCard**.
- Campos como **email**, **endereco**, **profissao**, etc., usam `|| ''` para evitar erros com **undefined**.

Documentação Técnica: Tela “Listar Pacientes”

 Caminho: `src/renderer/pages/Paciente/listar.tsx`

Objetivo da Tela

Apresentar, de forma organizada e filtrável, **todos os pacientes com um status específico** (**ativo**, **pausado**, **finalizado**), com foco em **interatividade e navegação para outras áreas do sistema**, como sessões, anamnese e financeiro.

Filtro de Listagem

- A filtragem é feita com base no parâmetro **status** da URL:

ts

Copiar código

```
const statusFiltro = searchParams.get('status') || 'ativo';
```

- Também há busca textual por nome (**busca**):

ts

Copiar código

```
pacientes.filter(p => p.status === statusFiltro &&  
p.nome_completo.toLowerCase().includes(busca.toLowerCase()))
```

Estrutura dos Cards Interativos

Cada paciente é representado por um **card com expansão colapsável**, que exibe opções de ação.

Card Colapsável

Elemento	Componente	Ação
 Nome	Typography	Mostra nome em destaque
 Telefone	Typography	Breve resumo do paciente
 Clique no Card	Collapse	Expande para mostrar ações

Ações no Card Expandido

Botão	Função
 Exibir dados completos	Abre um Modal com todos os dados do paciente
 Editar	Redireciona para <code>/Paciente/editar?id=...</code>

 Sessões	Redireciona para <code>/Paciente/sessoes?id=...</code>
 Anamnese	Redireciona para <code>/Anamnese?id=...</code>
 Financeiro	Redireciona para <code>/Financeiro?id=...</code>
 Relatórios	Redireciona para <code>/Relatorios?id=...</code>
 Apagar	Confirma e remove o paciente do banco e da lista renderizada

Estrutura de Componentes Utilizados

- `Paper` → Para o visual de cada card
 - `Collapse` → Para controlar a expansão do conteúdo
 - `Modal` → Para mostrar os **detalhes completos** do paciente
 - `Snackbar` e `Alert` → Para exibir mensagens de erro caso a busca falhe
-

Modal de Detalhes

- Exibido quando o botão “ Exibir dados completos” é clicado.
 - Apresenta nome, CPF, telefone, email, endereço, profissão, estado civil, status e observações.
 - Se o campo de observações for muito longo, exibe botão “Ler mais / Mostrar menos”.
-

Atualização da Lista após Exclusão

O paciente excluído é **removido da lista exibida dinamicamente** com:

ts

Copiar código

```
setPacientes(pacientes.filter(p => p.id !== item.id));
```

Estética

- Plano de fundo com `background3.png`
 - Cards com `Colors.roxo2gradient` e tipografia `Montserrat`
 - Responsividade e espaçamento cuidadoso com `Box`, `Container` e `Paper`
-

Considerações Finais

- Os **cards expandidos funcionam como um painel de controle do paciente**, permitindo acesso rápido a:
 - Sessões
 - Anamnese
 - Relatórios
 - Financeiro
 - Edição e exclusão
- A estrutura está **modular e escalável**, com navegação clara e componentes reutilizáveis.

Estrutura completa do banco, API, preload, telas e componentes.

1. Estrutura da Tabela – pacientes (database.ts)

CREATE TABLE IF NOT EXISTS pacientes (...)

Campo	Tipo	Obrigatório	Descrição
id	INTEGER (PK)	Sim	Identificador único autoincrementável
nome	TEXT	Sim	Nome completo do paciente
data_nascimento	TEXT	Não	Data de nascimento (string ISO)
cpf	TEXT	Não	Número do CPF
telefone	TEXT	Não	Número de telefone
email	TEXT	Não	E-mail
profissao	TEXT	Não	Profissão
estado_civil	TEXT	Não	Estado civil
sexo	TEXT	Não	⚠ Utilizado no frontend, mas ausente no banco
criado_em	TEXT	Não	Timestamp de criação (datetime local)
atualizado_em	TEXT	Não	Timestamp da última atualização

2. Interface de Paciente (Paciente.ts e globals.d.ts)

Campo	Tipo	Descrição
id	number	Identificador do paciente
nome	string	Nome completo
data_nascimento	string	Data de nascimento (ISO)
cpf	string	CPF
telefone	string	Telefone
email	string	E-mail
profissao	string	Profissão
estado_civil	string	Estado civil
criado_em	string	Data de criação
atualizado_em	string	Data de atualização

3. Funções de CRUD (src/db/paciente.ts)

- adicionarPaciente(paciente) → INSERT INTO
- listarPacientes() → SELECT *
- buscarPacientePorId(id) → SELECT WHERE id=?
- editarPaciente(paciente) → UPDATE SET

- `excluirPaciente(id) → DELETE FROM`

4. Serviços e IPC (Service, IPCMain, Preload)

- `pacienteService.ts` encapsula chamadas à `window.api.paciente`
- `ipcMain.handle` registra: listar, adicionar, editar, excluir, buscarPorId
- `preload.ts` expõe a API via `contextBridge` em `window.api`

5. Telas de Paciente (`index.tsx`, `adicionar.tsx`, `editar.tsx`, `listar.tsx`)

`index.tsx`: menu inicial com rotas para adicionar e listar

`adicionar.tsx`: formulário com campos mascarados e validação

`editar.tsx`: carrega paciente via id, atualiza backend

`listar.tsx`: cards com Collapse e ações → editar, sessões, anamnese, excluir

Ações no Card Expandido

Ação	Descrição
 Exibir dados completos	Abre um modal com todos os dados
 Editar	Redireciona para <code>/editar</code>
 Sessões	Vai para a tela de sessões do paciente
 Anamnese	Abre a anamnese do paciente
 Financeiro	Mostra o financeiro vinculado
 Relatórios	Mostra relatórios clínicos
 Apagar	Exclui paciente do banco e atualiza a tela

6. Componentes Reutilizáveis

Componente	Arquivo
<code>XPDateField</code>	<code>src/renderer/components/XPDateField.tsx</code>
<code>XPCPFField</code>	<code>src/renderer/components/XPCPFField.tsx</code>
<code>XPPhoneField</code>	<code>src/renderer/components/XPPhoneField.tsx</code>
<code>FormularioCard</code>	<code>src/renderer/components/FormularioCard.tsx</code>
<code>VoltarGlobal</code>	<code>src/renderer/components/VoltarGlobal.tsx</code>